GNC Project
Interview Preparation Guide

Complete Technical Deep-Dive

Mission: Miami → Moon (2 orbits) → Jupiter (3 orbits) → Miami

Duration: ∼4.1 years

Total ΔV: ∼10.5 km/s

Spacecraft Mass: 6,000 kg

Languages & Tools

Python	**C++**	**MATLAB/Simulink**	**YAML**
∼15,000 lines	∼3,000 lines	Verification	Configuration

February 5, 2026

Contents

1 Control Systems **2**
- 1.1 PID Controller (Classical) . . . 2
- 1.2 LQR Controller (Optimal) . . . 2
- 1.3 Sliding Mode Control (Robust) . . . 2
- 1.4 MPC (Model Predictive Control) . . . 3

2 Navigation & State Estimation **3**
- 2.1 Extended Kalman Filter (EKF) . . . 3
  - 2.1.1 Multiplicative Quaternion Error . . . 3
  - 2.1.2 Joseph Form Covariance Update . . . 3
- 2.2 Sensors Modeled . . . 4

3 Orbital Mechanics **4**
- 3.1 Key Algorithms . . . 4
  - 3.1.1 Hohmann Transfer . . . 4
  - 3.1.2 Bi-elliptic Transfer . . . 4
- 3.2 RK4 Integrator (C++) . . . 4

4 Real-Time Systems (C++) **4**
- 4.1 Flight Computer Architecture . . . 4
  - 4.1.1 Rate Monotonic Scheduling (RMS) . . . 5
  - 4.1.2 Triple Modular Redundancy (TMR) . . . 5
- 4.2 Memory Pool (Zero-Allocation) . . . 5

5 Debugging & Error Resolution **5**

6 Quick Reference Card **6**

1 Control Systems

1.1 PID Controller (Classical)

File: `control/attitude_control.py` – `PIDController` class

Purpose: Three-axis attitude control with proportional, integral, and derivative terms.

Key Equation

$$\boldsymbol{\tau} = K_p \cdot \mathbf{e} + K_i \cdot \int \mathbf{e}\, dt + K_d \cdot \frac{d\mathbf{e}}{dt}$$

Special Features:

- **Anti-windup:** Clamps integrator to prevent saturation
- **Derivative filtering:** Low-pass filter on derivative to reduce noise
- **Per-axis tuning:** Different gains for roll, pitch, yaw

Interview Answer

"I implemented a PID controller with anti-windup and derivative filtering because raw derivative amplifies high-frequency noise. The anti-windup prevents integrator saturation during large errors."

1.2 LQR Controller (Optimal)

File: `control/attitude_control.py` – `LQRController` class

Purpose: Minimizes a quadratic cost function to find optimal gains.

Key Equation

$$\text{Minimize: } J = \int_0^\infty \left(\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}\right) dt$$

$$\text{Feedback law: } \mathbf{u} = -K\mathbf{x}, \quad K = R^{-1} B^T P$$

Implementation Steps:

1. Build linearized state-space model (A, B matrices)
2. Solve the Continuous Algebraic Riccati Equation (CARE)
3. Compute optimal gain: $K = R^{-1} B^T P$

Interview Answer

"LQR provides optimal full-state feedback. I solve the Riccati equation iteratively since we don't have scipy.linalg in flight software. The Q matrix weights state errors, R weights control effort—I tuned these for 0.1° pointing with minimal fuel use."

1.3 Sliding Mode Control (Robust)

File: `control/attitude_control.py` – `SlidingModeController` class

Key Equation

$$\mathbf{s} = \boldsymbol{\omega}_{err} + \Lambda \cdot \boldsymbol{\theta}_{err}$$

$$\mathbf{u} = \mathbf{u}_{eq} + \mathbf{u}_{sw}, \quad \mathbf{u}_{sw} = -\eta \cdot \text{sat}(\mathbf{s}/\phi)$$

The `sat()` function replaces `sgn()` to reduce chattering.

Interview Answer

"Sliding mode is ideal for large-angle slews because it's inherently robust to bounded uncertainties. I use a boundary layer (ϕ) to prevent chattering—without it, the actuators would oscillate at high frequency."

1.4 MPC (Model Predictive Control)

File: `control/optimal_control.py` – `MPCController` class

Key Advantages:

- Handles constraints (torque limits, rate limits)
- Preview of reference trajectory
- Receding horizon adapts to changes

Interview Answer

"MPC is the most flexible controller because it naturally handles constraints. I pre-compute the prediction matrices for efficiency. The receding horizon means we re-plan every step, making it robust to disturbances."

2 Navigation & State Estimation

2.1 Extended Kalman Filter (EKF)

File: `navigation/ekf.py`

State Vector (15 states):

$$\mathbf{x} = \begin{bmatrix} \text{pos}(3) & \text{vel}(3) & \text{att_err}(3) & \text{gyro_bias}(3) & \text{accel_bias}(3) \end{bmatrix}^T$$

2.1.1 Multiplicative Quaternion Error

Why not put quaternion in state vector?

- Quaternion has 4 elements but only 3 DOF
- Unit-norm constraint violates Gaussian assumption
- **Solution:** Estimate small-angle error (3 states), maintain reference quaternion separately

2.1.2 Joseph Form Covariance Update

Key Equation

$$P = (I - KH)P(I - KH)^T + KRK^T$$

More numerically stable than $P = (I - KH)P$ for long-duration missions.

Interview Answer

"I use the multiplicative quaternion formulation because it respects the unit-norm constraint. After each update, I reset the attitude error by folding it into the reference quaternion. The Joseph form covariance update prevents numerical issues during long-duration missions."

2.2 Sensors Modeled

Sensor	Accuracy	Use Case
IMU (gyro)	0.1 mrad/s bias	Attitude rates (100 Hz)
Star Tracker	5 arcsec	Absolute attitude (10 Hz)
Sun Sensor	0.5°	Coarse attitude, eclipse detection
GPS	10 m / 0.1 m/s	Near-Earth navigation
Deep Space Network	5 m range	Jupiter cruise navigation

3 Orbital Mechanics

3.1 Key Algorithms

3.1.1 Hohmann Transfer

Key Equation

$$\Delta V_1 = \sqrt{\frac{\mu}{r_1}}\left(\sqrt{\frac{2r_2}{r_1+r_2}} - 1\right)$$

$$\Delta V_2 = \sqrt{\frac{\mu}{r_2}}\left(1 - \sqrt{\frac{2r_1}{r_1+r_2}}\right)$$

3.1.2 Bi-elliptic Transfer

Better than Hohmann when $r_2/r_1 > 11.94$

Interview Answer

"I implemented Hohmann for simple orbit raising and Lambert for arbitrary two-point boundary value problems. The Lambert solver is essential for interplanetary trajectory design."

3.2 RK4 Integrator (C++)

File: `dynamics_engine.cpp`

4th-order accuracy, 4 function evaluations per step:

```
k1 = f(t, y)
k2 = f(t + h/2, y + h*k1/2)
k3 = f(t + h/2, y + h*k2/2)
k4 = f(t + h, y + h*k3)
y_next = y + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
```

4 Real-Time Systems (C++)

4.1 Flight Computer Architecture

File: `flight_computer.h`

4.1.1 Rate Monotonic Scheduling (RMS)

Tasks with shorter periods get higher priority. Optimal for fixed-priority periodic tasks.

Schedulability bound: $U \leq n(2^{1/n} - 1)$

Task	Rate	Priority
Control loop	100 Hz	Highest
Navigation	10 Hz	Medium
Guidance	1 Hz	Lower
Telemetry	5 Hz	Lowest

4.1.2 Triple Modular Redundancy (TMR)

Run computation 3 times, take majority vote. Protects against radiation-induced bit flips (SEU).

Interview Answer

"I implemented rate-monotonic scheduling because it's provably optimal for periodic tasks. TMR provides radiation tolerance using COTS processors—we run critical calculations 3 times and vote."

4.2 Memory Pool (Zero-Allocation)

File: `memory_pool.h`

Why no malloc in real-time?

- Non-deterministic timing
- Fragmentation
- Page faults

```
template<typename T, size_t N>
class MemoryPool {
    alignas(64) T blocks_[N];   // Cache-aligned
    T* free_list_;              // Intrusive linked list
};
```

Interview Answer

"Dynamic memory allocation is forbidden in flight software because malloc has non-deterministic timing. I use pre-allocated memory pools with O(1) allocate/free. The blocks are cache-line aligned for optimal performance."

5 Debugging & Error Resolution

Error 1: PD Controller Sign Error

Symptom: Attitude diverged—spacecraft spun out of control.
Root Cause: Wrong quaternion error convention. Using $q_{err} = q_{current} \times q_{target}^{-1}$ instead of $q_{err} = q_{target} \times q_{current}^{-1}$.
Fix: Standardized on $q_{err} = q_{target} \times q_{current}^{-1}$ with scalar-positive enforcement.

⚠ Error 2: Quaternion Normalization Drift

Symptom: After many integration steps, quaternion norm drifted from 1.0.
Root Cause: Numerical integration accumulates floating-point errors.
Fix: Renormalize after every step + multiplicative EKF formulation.

⚠ Error 3: Integrator Windup

Symptom: Controller output saturated and stayed saturated.
Root Cause: Unbounded integrator accumulated huge value.
Fix: Anti-windup clamping: `integral = clip(integral, -limit, limit)`

⚠ Error 4: malloc() Timing Jitter

Symptom: Control loop timing varied from 8ms to 50ms.
Root Cause: Dynamic allocation has non-deterministic timing.
Fix: Pre-allocated memory pools, eliminated all dynamic allocation.

⚠ Error 5: EKF Numerical Instability

Symptom: Covariance matrix became non-positive-definite after days.
Root Cause: Standard update $P = (I - KH)P$ is numerically unstable.
Fix: Joseph form: $P = (I - KH)P(I - KH)^T + KRK^T$

⚠ Error 6: Propellant Budget Exceeded

Symptom: Mission requires ∼10.5 km/s but only ∼2 km/s available.
Root Cause: Direct transfer to Jupiter isn't mass-feasible with chemical propulsion.
Fix: Proposed VEEGA gravity assists, documented constraint.

6 Quick Reference Card

Equations to Know

Rocket Equation: $\Delta V = I_{sp} \cdot g_0 \cdot \ln(m_0/m_f)$

Vis-Viva: $v^2 = \mu \left(\frac{2}{r} - \frac{1}{a} \right)$

Quaternion Kinematics: $\dot{q} = \frac{1}{2} q \otimes \omega_q$

Euler's Equations: $I\dot{\omega} = \tau - \omega \times (I\omega)$

Kalman Gain: $K = PH^T(HPH^T + R)^{-1}$

Good Luck Tomorrow!

Remember: You built this. You understand it. You debugged it.
Be confident.